



Digital Signal Processing - Term Project: Fun With Filters

Digital Signal Processing

Instructor: Aggelos Blestas

Location: ARC 103

Exercise 1: Fun With FIR Filtering

1.2: Frequency Response (Order 64, Low-Pass)

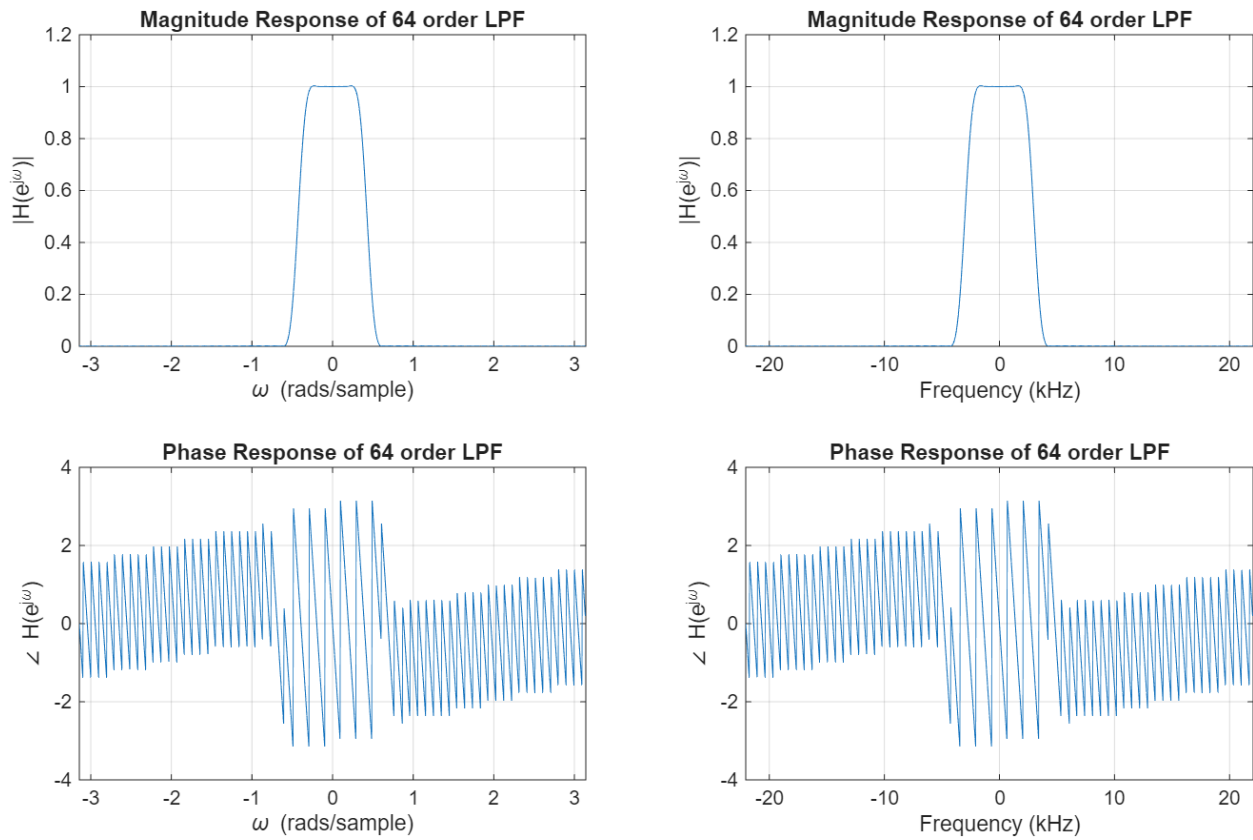


Figure 1: Magnitude and phase response of the 64-order FIR low-pass filter (cutoff 3 kHz).

1.3: Frequency Response (Order 32, Low-Pass)

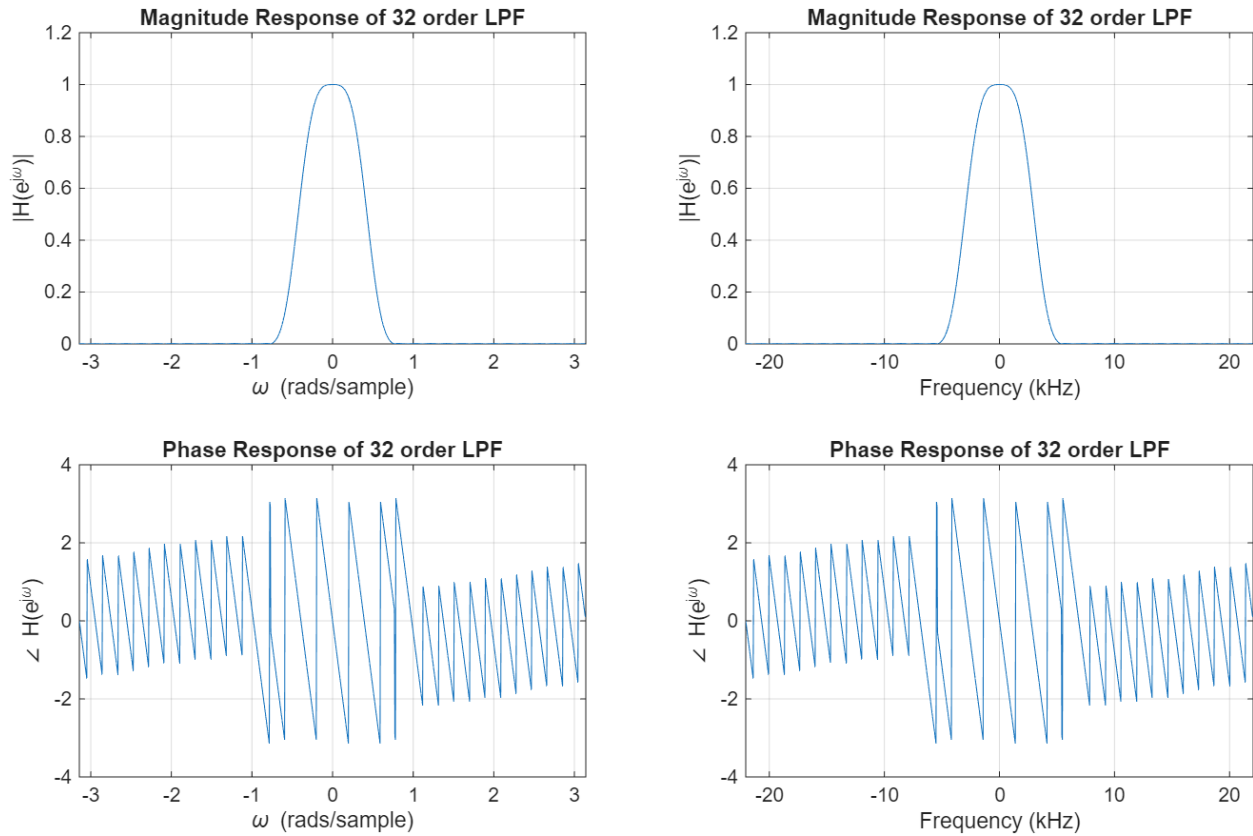


Figure 2: Magnitude and phase response of the 32-order FIR low-pass filter (cutoff 3 kHz).

Compared to the 64-order filter, the 32-order filter exhibits a wider and more gradual transition band. The rolloff from passband to stopband is noticeably less steep, meaning frequencies near the 3 kHz cutoff are not as sharply attenuated. This is because a lower filter order means fewer coefficients, which limits the filter's ability to approximate an ideal rectangular frequency response. The phase response of the 32-order filter also shows fewer oscillation cycles across the same frequency range, which makes sense since a lower order filter introduces less delay and less delay means the phase changes more slowly with frequency.

1.4: Frequency Response (Order 64, High-Pass)

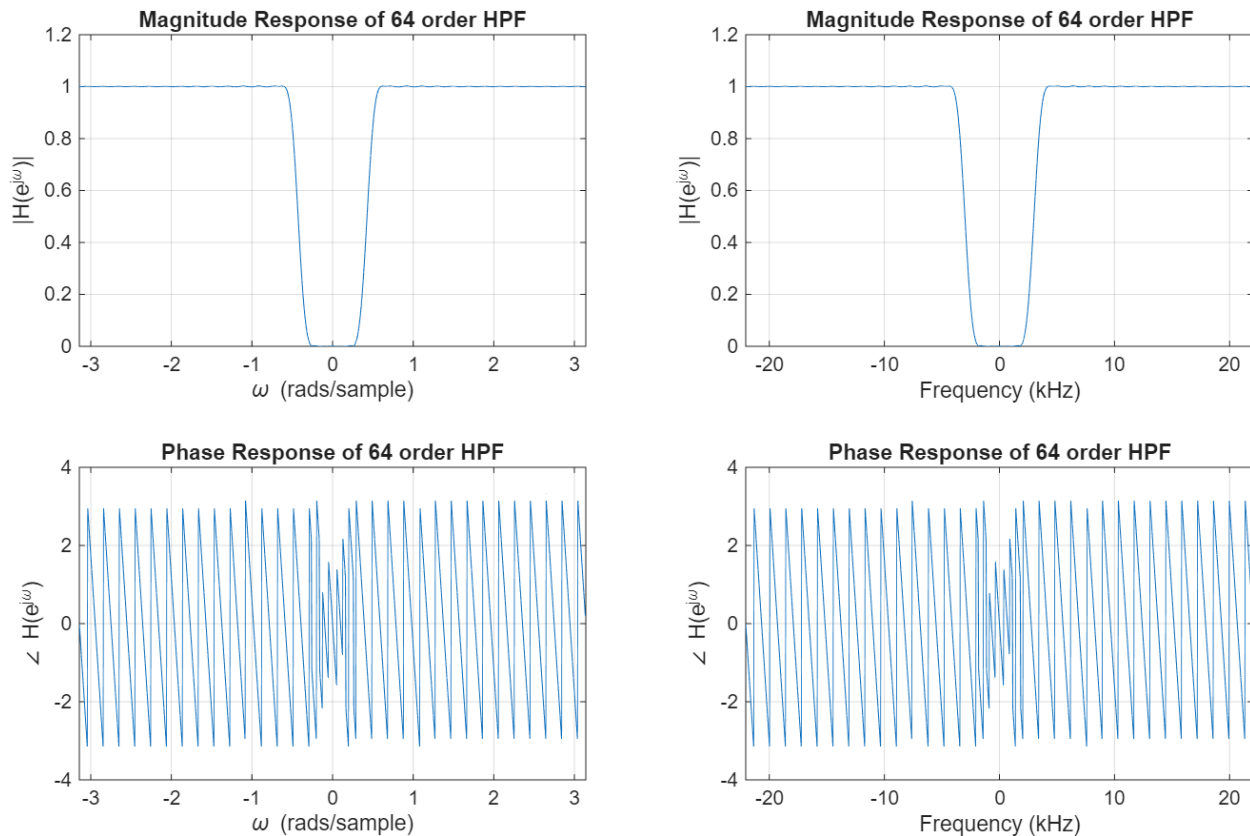


Figure 3: Magnitude and phase response of the 64-order FIR high-pass filter (cutoff 3 kHz).

Exercise 2: Fun With Fingerprinting Identification

2.1: WAV Files

Instead of recording audio files with our phones, we chose to use existing audio of songs we already had. This allowed us to use the entire song in our database for testing our comparison algorithm, instead of just comparing two small chunks of exactly the same sample. We used the following .WAV files (converted from .mp3):

1. Alice Cooper - School's Out.wav
2. Cream - Sunshine of Your Love.wav
3. DRAGONFORCE - Through the Fire and Flames.wav
4. Foghat - Slow Ride.wav
5. Guitar Battle vs Tom Morello.wav
6. Heart - Barracuda.wav
7. KISS - Rock and Roll All Nite.wav
8. Lynnyrd Skynyrd - Sweet Home Alabama.wav
9. Mountain - Mississippi Queen.wav

10. Pat Benatar - Hit Me With Your Best Shot.wav
11. Poison - Talk Dirty to Me.wav
12. Rage Against the Machine - Bulls on Parade.wav
13. Social Distortion - Story of my Life.wav

The songs are a collection of music from the video game *Guitar Hero III: Legends of Rock*.

We wrote the following python program to slice a short 3 second chunk from each audio file:

```
1 import numpy as np
2 import librosa
3 import soundfile as sf
4 from pathlib import Path
5
6
7 def extract_chunks(input_dir=".", start_s=10, duration_sec=3):
8     base_path = Path(input_dir)
9     mp3_files = list(base_path.glob("*.mp3"))
10
11     if not mp3_files:
12         print("No MP3 files found in the directory.")
13         return
14
15     print(f"Found {len(mp3_files)} files. Starting extraction...")
16
17     for mp3_path in mp3_files:
18         try:
19             # Load audio (keeps original sample rate)
20             y, sr = librosa.load(mp3_path, sr=None, mono=False)
21
22             # Convert duration to samples
23             num_samples = int(duration_sec * sr)
24
25             # Start time in sample index
26             start_sample = int(start_s * sr)
27
28             # Handle mono vs stereo
29             if y.ndim == 1:
30                 chunk = y[start_sample : start_sample + num_samples]
31             else:
32                 chunk = y[:, start_sample : start_sample + num_samples]
33
34             output_filename = mp3_path.with_suffix(".wav")
35
36             # Save WAV
37             sf.write(output_filename, chunk.T if y.ndim > 1 else chunk, sr)
38
39             print(f"Successfully saved: {output_filename}")
40
41         except Exception as e:
42             print(f"Error processing {mp3_path.name}: {e}")
43
44
45 if __name__ == "__main__":
46     extract_chunks(input_dir=".")
```

2.2: What is fingerprinting?

The idea of fingerprinting is to create a unique identifier for a song based on its sound characteristics. By extracting a spectrogram from an audio file, we can create a “fingerprint” vector using some of the data from the spectrogram. In the provided code, we first compute the spectrogram of the audio file. If we were to just take an FFT of the input signal, we would see which frequencies exist in the signal, but not when they occur. Instead, when using a spectrogram, a window slides across the time-domain signal, takes an FFT at each position, and then stacks the results into a N-by-1 vector. This produces a 2D time-frequency representation of the audio sample. The columns represent every FFT window, and the rows represent the magnitude of a frequency in that specific window. This array tells us how much of each frequency is present at each moment in time.

The next step in constructing a fingerprint is peak detection, where we find the frequencies with the largest magnitude in the spectrogram. For every point $S(f, t)$, look in the surrounding 7×7 neighborhood and ask two questions:

1. Is $S(f, t)$ the maximum value in that neighborhood?
2. Is $S(f, t)$ above the amplitude threshold of 10?

If both of these conditions are true, then that specific point is marked as a peak.

The last step is hashing, where the peaks are converted into fingerprint hashes. We currently have a list of (f, t) coordinates, but if stored directly they would be fragile. If the same recording started a second later, then all the t values would be shifted and it would be difficult to match the two signals. Instead of storing all individual peaks, we store the relationships between pairs of peaks. For each peak (we called them “anchors”), you pair it with the next `fanOut=5` peaks (defined in MATLAB code) in the list and record:

$$[f_1 \quad f_2 \quad \Delta t \quad t_1]$$

Where,

- f_1 — frequency of the anchor peak
- f_2 — frequency of the partner peak
- $\Delta t = t_2 - t_1$ — time difference between the two peaks
- t_1 — absolute time of the anchor

The actual fingerprint hash is the triplet $(f_1, f_2, \Delta t)$. The t_1 is stored alongside it but is only used later for alignment, not for matching. Each song’s complete set of hashes constitutes its fingerprint, stored in a database.

The fingerprint is robust to noise because the strongest spectral peaks are usually much larger than the background noise, and will therefore survive background interference; a noisy recording of the same song will still produce many of the same dominant peaks, and therefore many of the same hash triplets.

The use of relationships between peaks (via Δt) rather than absolute timestamps also makes the fingerprint invariant to recording start time: if the query clip begins 2 seconds into the song, the time differences between peaks remain unchanged.

2.3a: Algorithmic Fingerprinting Comparison

Our identification algorithm works in three stages:

1. **Database construction:** The fingerprinting code is run on all 13 songs, and each song’s hashes are stored in a cell array saved to `database.mat`.
2. **Anchor matching:** Given a query WAV file, its hashes are computed. The first hash of the query is used as an anchor. All entries in each database song’s hash table that match this anchor within a frequency tolerance ϵ_f and time tolerance ϵ_t are identified as candidates.

3. **Similarity scoring:** For each candidate match, a time offset $\Delta t_{\text{offset}} = t_{1,\text{db}} - t_{1,\text{query}}$ is computed. Every remaining query hash is then shifted by this offset and searched for in the database. The number of query hashes that find a match within tolerance is the similarity score for that candidate. The best score across all candidates is taken as the song's final score. The song with the highest score is returned as the match.

Comparison Metric

A fingerprint is a $[N \times 4]$ matrix. N represents the number of hashes in the fingerprint. Typical fingerprint:

$$\begin{bmatrix} f_{1,1} & f_{2,1} & \Delta t_1 & t_{1,1} \\ f_{1,2} & f_{2,2} & \Delta t_2 & t_{1,2} \\ f_{1,3} & f_{2,3} & \Delta t_3 & t_{1,3} \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

The order of the hashes encodes the information we need to identify the song. The comparison should be the number of matching hashes in the sample fingerprint with the database fingerprint.

Hashes should have the same order and similar values in a song passed through the same algorithm which generates the fingerprints.

First compare the difference between the hash of $N=1$ of the sample with all the samples in your database.

A comparison is:

$$\begin{bmatrix} f_{1,N=1} \\ f_{2,N=1} \\ \Delta t_{N=1} \\ t_{1,N=1} \end{bmatrix} - \begin{bmatrix} f_{1,\text{Database}} \\ f_{2,\text{Database}} \\ \Delta t_{\text{Database}} \\ t_{1,\text{Database}} \end{bmatrix} = \begin{bmatrix} f_{1,\text{Error}} \\ f_{2,\text{Error}} \\ \Delta t_{\text{Error}} \\ t_{1,\text{Error}} \end{bmatrix}$$

The similarity metric is a count of temporally consistent hash matches. For each query hash $q = (f_1, f_2, \Delta T, t_1)$, a database hash d is considered a match if:

$$|f_{1,q} - f_{1,d}| < \epsilon_f \quad (1)$$

$$|f_{2,q} - f_{2,d}| < \epsilon_f \quad (2)$$

$$|\Delta T_q - \Delta T_d| < \epsilon_t \quad (3)$$

$$|t_{1,q} + \Delta t_{\text{offset}} - t_{1,d}| < \epsilon_t \quad (4)$$

Where we use $\epsilon_f = 50$ Hz and $\epsilon_t = 0.05$ seconds. The fourth condition is what enforces temporal consistency. A match in the wrong part of the song is rejected, since the time offset computed from the anchor fixes where in the database each subsequent query hash should appear. This makes the metric much more discriminative than a simple hash lookup, as coincidental frequency matches that occur at the wrong time are excluded.

We used this as a condition instead of doing a sequential pairwise comparison where, once you find the anchor, walk forward through both the database and query in lockstep and compare them one by one. This is because if the database or query has even just one missing/additional hash, the total score calculation would be poor.

2.3b: Matlab Implementation

The algorithm is implemented across three files: `computeHashes.m`, `buildDatabase.m`, and `identifySong.m`.

`computeHashes.m` encapsulates the full fingerprinting pipeline from the provided sample code into a reusable function. In order to be thorough in our analysis, we used the Communications Toolbox MATLAB function `awgn` to add white Gaussian Noise to the sample to be identified.

```
1 function hashes = computeHashes(filename)
2     windowSize = 4096;
3     overlap = 2048;
```

```

4     fanOut = 5;
5     ampMin = 10;
6     neighborhoodSize = 3;
7
8     [x, Fs] = audioread(filename);
9     x = mean(x, 2);
10
11     % Add noise to wav
12     x = awgn(x, 10, 'measured');
13
14     [S, F, T] = spectrogram(x, windowSize, overlap, [], Fs);
15     S = abs(S);
16
17     peaks = zeros(size(S));
18     [nFreqBins, nTimeBins] = size(S);
19
20     for t = (1 + neighborhoodSize):(nTimeBins - neighborhoodSize)
21         for f = (1 + neighborhoodSize):(nFreqBins - neighborhoodSize)
22             localWindow = S(f - neighborhoodSize:f + neighborhoodSize, ...
23                             t - neighborhoodSize:t + neighborhoodSize);
24             if S(f, t) == max(localWindow(:)) && S(f, t) > ampMin
25                 peaks(f, t) = 1;
26             end
27         end
28     end
29
30     [peakFreqIdx, peakTimeIdx] = find(peaks);
31
32     hashes = [];
33     for i = 1:length(peakTimeIdx)
34         for j = 1:fanOut
35             if i + j <= length(peakTimeIdx)
36                 t1 = T(peakTimeIdx(i));
37                 t2 = T(peakTimeIdx(i + j));
38                 f1 = F(peakFreqIdx(i));
39                 f2 = F(peakFreqIdx(i + j));
40                 deltaT = round(t2 - t1, 2);
41                 hashes(end+1, :) = [round(f1), round(f2), deltaT, round(t1
42                                     , 2)];
43             end
44         end
45     end

```

buildDatabase.m processes all 13 songs and saves their hashes:

```

1     base_dir = "wav_source";
2     songs = {"Alice Cooper - School's Out.wav", "Cream - Sunshine of Your Love
3             .wav", ...
4             "DRAGONFORCE - Through the Fire and Flames.wav", "Foghat - Slow Ride.
5             wav", "Guitar Battle vs Tom Morello.wav", "Heart - Barracuda.wav",
6             ...
7             "KISS - Rock and Roll All Nite.wav", "Lynyrd Skynyrd - Sweet Home
8             Alabama.wav", "Mountain - Mississippi Queen.wav", ...

```

```

5      "Pat Benatar - Hit Me With Your Best Shot.wav", "Poison - Talk Dirty
      to Me.wav", "Rage Against the Machine - Bulls on Parade.wav"...
6      "Social Distortion - Story of my Life.wav"};
7
8      database = cell(length(songs), 1); % each cell holds the hashes for one
      song
9
10     for s = 1:length(songs)
11         fprintf('Processing song %d: %s\n', s, fullfile(base_dir, songs{s}));
12         database{s} = computeHashes(fullfile(base_dir, songs{s}));
13     end
14
15     save('database.mat', 'database', 'songs');
16     disp('Database saved.');
```

identifySong.m identifies a query WAV against the database:

```

1      load('database.mat'); % loads database and songs
2
3      queryFile = 'Foghat - Slow Ride.wav'; %input song we are comparing to
      database
4      root_dir = 'wav_chunks';
5      queryHashes = computeHashes(fullfile(root_dir, queryFile)); %compute input
      songs hashes
6
7      % These are tolerances because some of these frequency calculations wont
      be
8      % exactly the same even if its the same song
9      freq_tol = 50; % Hz
10     time_tol = 0.05; % seconds
11
12     %iterate through the 13 songs in the database
13     scores = zeros(13, 1);
14     for i = 1:13
15         scores(i) = computeSimilarity(queryHashes, database{i}, freq_tol,
            time_tol); %find highest score for each song in database
16     end
17
18     [bestScore, bestIdx] = max(scores); %find max score out of score array
19     fprintf('Best match: song %d (%s) with score %d\n', bestIdx, songs{bestIdx
        }, bestScore);
20
21     % show all scores
22     disp('All scores:');
23     for i = 1:13
24         fprintf(' Song %d (%s): %d matches\n', i, songs{i}, scores(i));
25     end
```

computeSimilarity.m implements the anchor-based scoring:

```

1      function score = computeSimilarity(queryHashes, dbHashes, freq_tol,
      time_tol)
2          score = 0;
3
4          % grab the first query hash as anchor
5          q_anchor = queryHashes(1, :); % [f1, f2, dT, t1]
```

```

6
7     % find all candidates in dbHashes that match the first hash entry
      within a tolerance
8     candidateIdx = find( ...
9         abs(dbHashes(:,1) - q_anchor(1)) < freq_tol & ...
10        abs(dbHashes(:,2) - q_anchor(2)) < freq_tol & ...
11        abs(dbHashes(:,3) - q_anchor(3)) < time_tol );
12
13     if isempty(candidateIdx)
14         return; % no candidates, return score = 0 and exit
15     end
16
17     % for each candidate, compute the time offset and score
18     for c = 1:length(candidateIdx)
19         t_offset = dbHashes(candidateIdx(c), 4) - q_anchor(4);
20
21         % now count how many query hashes match dbHashes at this offset
22         candidateScore = 0;
23         for i = 1:size(queryHashes, 1)
24             q = queryHashes(i, :);
25             % shift query time (t1) by offset to align with database
26             q_t1_shifted = q(4) + t_offset;
27
28             %match checks if any row in dbHashes matches all 4 hash values
29             %within a tolerance. if it does, it sets match = true.
30             match = any( ...
31                 abs(dbHashes(:,1) - q(1)) < freq_tol & ...
32                 abs(dbHashes(:,2) - q(2)) < freq_tol & ...
33                 abs(dbHashes(:,3) - q(3)) < time_tol & ...
34                 abs(dbHashes(:,4) - q_t1_shifted) < time_tol );
35
36             if match
37                 candidateScore = candidateScore + 1;
38             end
39         end
40
41         % keep the best score across all candidates
42         score = max(score, candidateScore);
43     end
44 end

```

Justification

The algorithm works well for several reasons. First, the use of spectral peaks rather than raw spectrogram values makes the fingerprint compact and noise-robust. Only the most energetically dominant time-frequency points survive, and these are stable across different recordings of the same audio. Second, the hash triplet $(f_1, f_2, \Delta t)$ encodes a *relational* property of the audio rather than an absolute one, making it invariant to recording start time and volume. Third, the time offset mechanism ensures that only *temporally consistent* matches contribute to the score. A database song that happens to share a few frequencies with the query at random times will score near zero, while the correct song will produce a large cluster of consistent matches. Together, these properties make the algorithm reliable even for short, slightly noisy query clips.

Results

The algorithm was tested by querying a 3 second clip of *Slow Ride* by Foghat against a database of 13 songs. The sound clip has been modified to have +70dB of white Gaussian noise to simulate a busy environment. The results show that the algorithm is still able to correctly identify the clip from the database. The output was as follows:

Best match: song 4 (Foghat - Slow Ride.wav) with score 338

All scores:

Song 1 (Alice Cooper - School's Out.wav): 47 matches
Song 2 (Cream - Sunshine of Your Love.wav): 59 matches
Song 3 (DRAGONFORCE - Through the Fire and Flames.wav): 54 matches
Song 4 (Foghat - Slow Ride.wav): 338 matches
Song 5 (Guitar Battle vs Tom Morello.wav): 76 matches
Song 6 (Heart - Barracuda.wav): 55 matches
Song 7 (KISS - Rock and Roll All Nite.wav): 66 matches
Song 8 (Lynyrd Skynyrd - Sweet Home Alabama.wav): 81 matches
Song 9 (Mountain - Mississippi Queen.wav): 86 matches
Song 10 (Pat Benatar - Hit Me With Your Best Shot.wav): 67 matches
Song 11 (Poison - Talk Dirty to Me.wav): 56 matches
Song 12 (Rage Against the Machine - Bulls on Parade.wav): 56 matches
Song 13 (Social Distortion - Story of my Life.wav): 76 matches

The correct song was identified with a score of 338 matches, compared to the next highest score of 86 for *Mississippi Queen* by Mountain. This large margin of nearly 4× the second-place score demonstrates the effectiveness of the temporal consistency requirement in the similarity metric. The non-matching songs score much lower because, while they may share occasional frequency content with the query clip, their hashes do not align consistently with the computed time offset and are therefore rejected.

Contributions

Hans

- Completed Problem 1 and answered relevant questions
- Contributed to writing paragraphs on what fingerprinting is
- Wrote section 2.3b
- Contributed to writing fingerprint matching algorithm matlab code

Ryan

- Saved 13 WAV recordings
- Contributed to writing paragraphs on what fingerprinting is
- Wrote section 2.3a
- Contributed to writing fingerprint matching algorithm matlab code
- Formatted complete Latex document